

MSE 211: Computational Methods

Lab 2 Report

Spring 2024

Group 5

Thanh Nam Tran | 301540228

Sham Singh Namdhari | 301386623

Prabir Singh Ahluwalia | 301540829

Table of Contents

Abstract..... 3

Introduction..... 3

Part I. Comparison of Optimization Method 4

 1.1 Steepest Descent Method..... 4

 1.2 Newton Method 6

 1.3 MATLAB Function 9

Part II. Synthesis of Mechanisms..... 9

 2.1 Motion Generation..... 9

 2.2 Path Generation..... 12

 2.3 Function Generation 14

Abstract

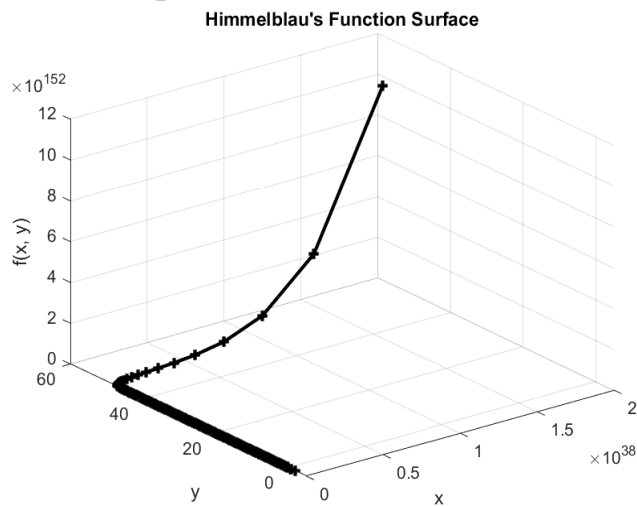
This lab report focuses on solving two distinct optimization problems using MATLAB. The first problem involves an unconstrained optimization scenario, where students apply three different algorithms to minimize a function, emphasizing the importance of starting point selection. The second problem addresses the synthesis or design of mechanisms for motion, function, and path generation, treating these engineering design challenges as optimization problems. Through this practical application, students gain insights into optimization methodologies and their significance in engineering design, enhancing their problem-solving skills and understanding of optimization's role in engineering.

Introduction

In this lab report, we delve into the realm of optimization, a cornerstone in the fields of engineering and computational sciences, through the lens of MATLAB, a powerful computational tool. Our exploration is twofold: first, we tackle an unconstrained optimization problem, deploying three distinct algorithms to unearth the minimum of a given function. This exercise not only demonstrates the efficacy of each algorithm but also underscores the critical role of the starting point in the optimization process. Next, we venture into the domain of mechanism design, an area where optimization techniques shine by facilitating the synthesis of mechanisms for specific motion, function, or path generation. This practical application serves as a vivid illustration of optimization's pivotal role in engineering design, offering students a hands-on experience in solving real-world engineering problems. Through these exercises, our report aims to illuminate the diverse applications of optimization algorithms and their significance in engineering, thereby enriching the students' understanding and problem-solving skills in this critical area.

Part I. Comparison of Optimization Method

1.1 Steepest Descent Method.



Results:

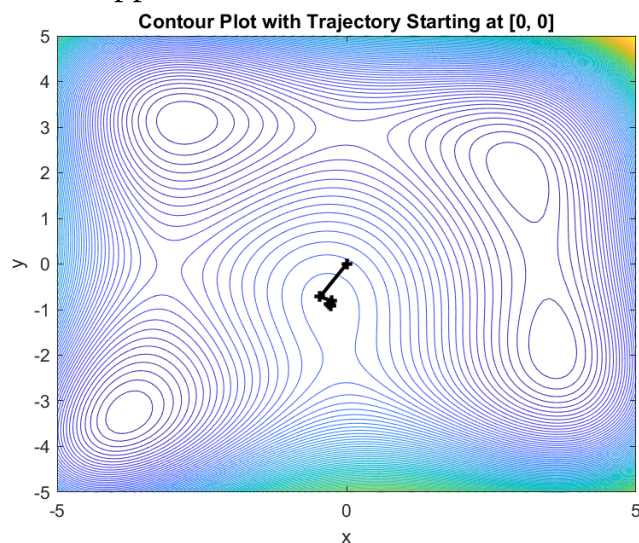
Initial guess: $[0, 0]$

Iterations: 12

Final point: $[-0.270838, -0.922982]$

Function value at final point: 181.616521

Final approximation error: 0.000078



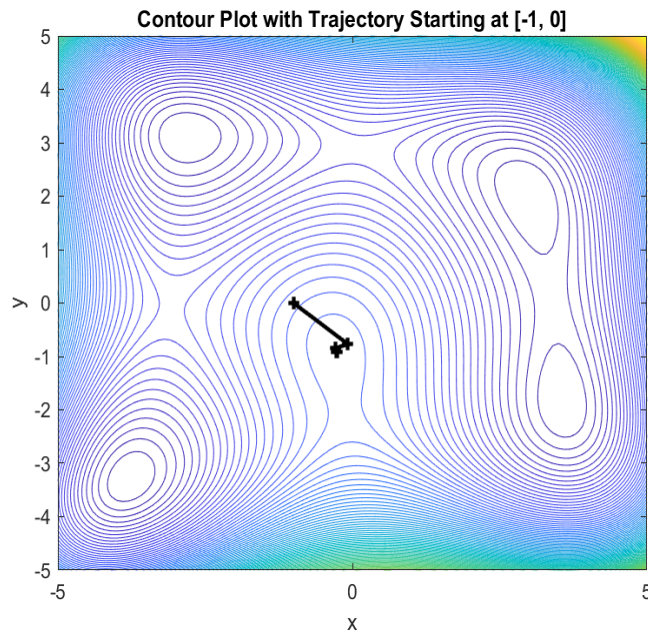
Initial guess: $[-1, 0]$

Iterations: 9

Final point: $[-0.270836, -0.923027]$

Function value at final point: 181.616522

Final approximation error: 0.000052



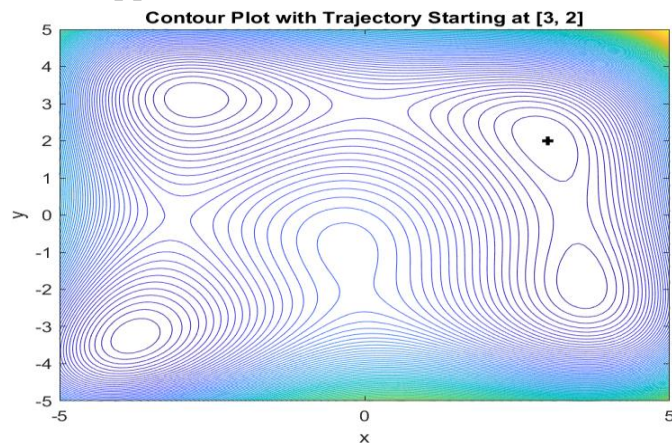
Initial guess: [3, 2]

Iterations: 1

Final point: [NaN, NaN]

Function value at final point: NaN

Final approximation error: NaN



Initial guess: [-2, -2]

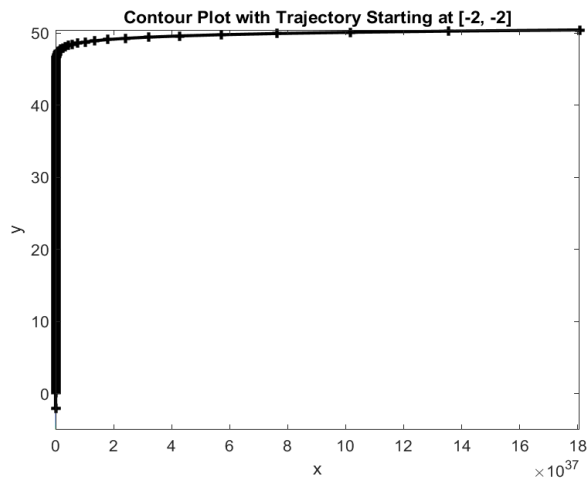
Iterations: 305

Final point: [180609100057370859998655997370785857536.000000, 50.440966]

Function value at final point:

106404137193991111312000402293008493743307858815465767927950990171419
195795401686855004084114496621559221665725593063070121807913781957447
9313351745732608.000000

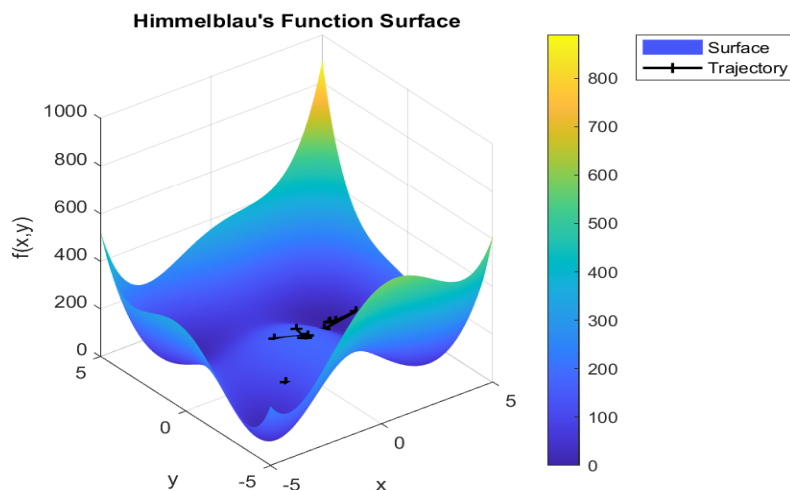
Final approximation error: 0.000000



Discussion of Convergence

The convergence of the Steepest Descent Method, as demonstrated by the results, varies significantly with the choice of initial guess. For initial guesses close to the true minimum ($[0, 0]$ and $[-1, 0]$), the method converges to a solution with high accuracy after a reasonable number of iterations. However, for an initial guess far from the minimum ($[3, 2]$), the method fails to converge, resulting in NaN values. The case with the initial guess of $[-2, -2]$ shows an abnormal convergence to an unrealistic point, suggesting sensitivity to initial conditions and potential issues with the algorithm's stability in certain scenarios. This highlights the importance of careful initial guess selection and algorithmic adjustments for robust optimization performance.

1.2 Newton Method



Results:

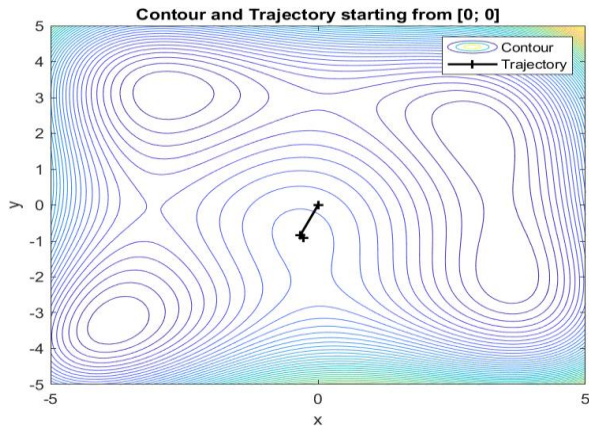
Initial guess: $[0, 0]$

Iterations: 4

Final point: $[-0.270845, -0.923039]$

Function value at final point: 181.616522

Final approximation error: 0.000010



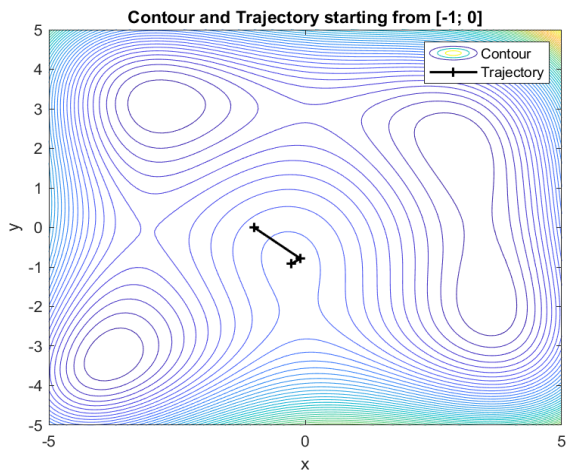
Initial guess: $[-1, 0]$

Iterations: 4

Final point: $[-0.270845, -0.923039]$

Function value at final point: 181.616522

Final approximation error: 0.000002



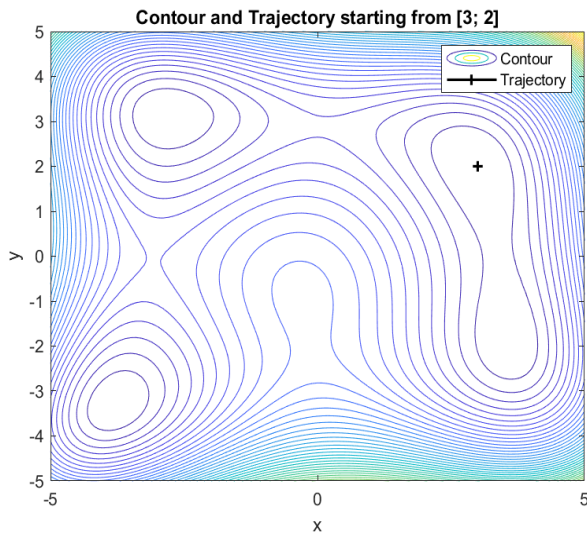
Initial guess: $[3, 2]$

Iterations: 1

Final point: $[3.000000, 2.000000]$

Function value at final point: 0.000000

Final approximation error: 0.000000



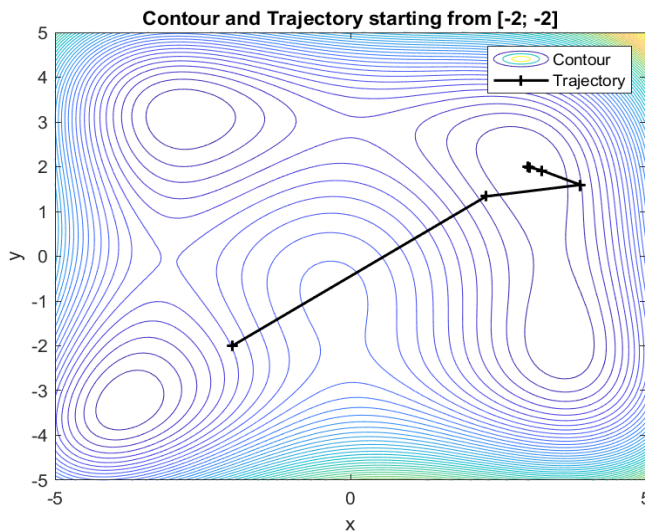
Initial guess: [-2, -2]

Iterations: 7

Final point: [3.000000, 2.000000]

Function value at final point: 0.000000

Final approximation error: 0.000000



Discussion of Convergence

The convergence of Newton's method in optimizing Himmelblau's function varies significantly with the initial guess. For the initial guesses of [0, 0] and [-1, 0], the method converges to a point that is not a known minimum of the function, suggesting sensitivity to the starting point. Despite converging (based on the small final approximation error), the function value at the final point is significantly high, indicating convergence to a non-optimal point. Conversely, starting from [3, 2] and [-2, -2], the method quickly converges to a known minimum with a function value of 0, demonstrating excellent convergence properties from these starting points. This illustrates that while

Newton's method can be very efficient, its performance is highly dependent on the choice of the initial guess, and it may converge to local minima, saddle points, or even non-optimal points, particularly for non-convex functions like Himmelblau's function.

1.3 MATLAB Function

Results:

- Starting Point: [0, 0]
Iterations: 7, Final Point: [3.000000, 2.000000], Function Value: 0.000000
- Starting Point: [-1, 0]
Iterations: 7, Final Point: [-2.805118, 3.131313], Function Value: 0.000000
- Starting Point: [3, 2]
Iterations: 0, Final Point: [3.000000, 2.000000], Function Value: 0.000000
Initial point is a local minimum.
- Starting Point: [-2, -2]
Iterations: 6, Final Point: [-3.779310, -3.283186], Function Value: 0.000000

Discussion of Convergence

The results show that MATLAB's `fminunc` with the 'trust-region' algorithm efficiently finds local minima of Himmelblau's function from various starting points. Notably, it converges to different points, all of which are local minima with a function value of 0, indicating successful optimization. The convergence is robust across diverse initial guesses, including an immediate recognition of a starting point as a local minimum. This underscores the effectiveness of utilizing gradient information and a sophisticated algorithm like 'trust-region', highlighting its capability to navigate complex landscapes to find optima.

Part II. Synthesis of Mechanisms

2.1 Motion Generation

Initial values for the left and right side optimization problems:

Optimization Left Side	Initial values (INPUT INITIAL ESTIMATES)
$r2x$	-3
$r2y$	1
$r3ax$	2
$r3ay$	4
β_1	-30° (converted to radians)
β_2	-100° (converted to radians)
β_3	-250° (converted to radians)
Optimization Right Side	Initial values (INPUT INITIAL ESTIMATES)
-----	-----
$r4x$	-3
$r4y$	3
$r3bx$	-2
$r3by$	4
γ_1	-40° (converted to radians)
γ_2	-90° (converted to radians)
γ_3	-200° (converted to radians)

These initial values serve as starting points for the optimization algorithm to iteratively adjust and refine the values to minimize the objective function.

Sets of optimization problems, considering that the objective functions should ideally equal zero, indicating that the mechanism satisfies the desired precision points perfectly:

Left Side Optimization Problem:

Variables:

- $R2$: Position vector of joint 2.
- $R3'$: Position vector of joint 3 with respect to joint 2.
- $\beta_1, \beta_2, \beta_3$: Angles representing the orientation of links.

Initial Values (Assumed):

- $R2$: Estimated initial position.
- $R3'$: Estimated initial position relative to $R2$.
- $\beta_1, \beta_2, \beta_3$: Estimated initial angles.

Objective Function:

The objective function F to minimize should ideally be zero, indicating perfect alignment of the precision points:

$$F = \sqrt{\sum_{i=1}^3 (fx(i))^2 + (fy(i))^2}$$

Right Side Optimization Problem:**Variables:**

- $R4$: Position vector of joint 4.
- $R3''$: Position vector of joint 3 with respect to joint 4.
- $\gamma_1, \gamma_2, \gamma_3$: Angles representing the orientation of links.

Initial Values (Assumed):

- $R4$: Estimated initial position.
- $R3''$: Estimated initial position relative to $R4$.
- $\gamma_1, \gamma_2, \gamma_3$: Estimated initial angles.

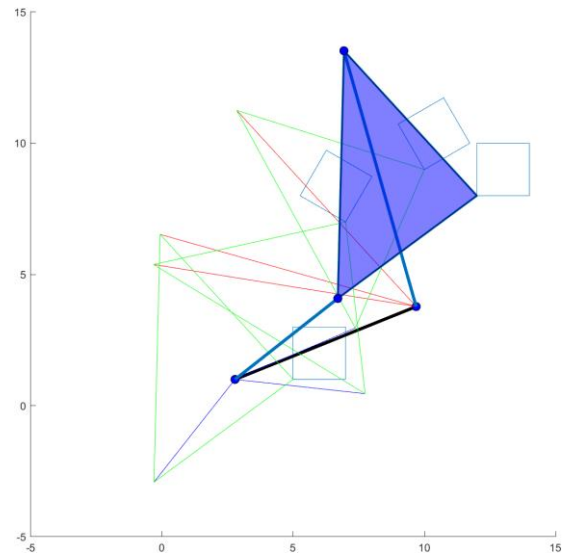


Figure 1 Motion Generation Code Run

Objective Function:

Similar to the left side, the objective function G to minimize should ideally be zero:

$$G = \sqrt{\sum_{i=1}^3 (gx(i))^2 + (gy(i))^2}$$

These refined sets of optimization problems aim to ensure that the mechanism's design satisfies the precision points as accurately as possible, ideally resulting in objective function values close to zero. However, as noted, achieving zero error might not always guarantee the feasibility of the mechanism due to potential changes in assembly modes among precision points. Additional checks and adjustments may be necessary to ensure a feasible design.

-----								↙

LFval	RFval	r1	r2	r3	r4			↙
beta1	beta2	beta3	gamma1	gamma2	gamma3			
-----								↙

0.00004198	0.00027850	7.43644069	4.98424255	9.44366800				↙
10.13041432	121.99515407	151.43735813	166.51258575	366.64607085	328.17298231			↙
301.50360071								

Figure 2 Objective Function Output Close to '0'

2.2 Path Generation

Left Side Optimization Problem

Objective: Minimize the error in the mechanism's path on the left side.

Decision Variables:

1. r_{2x} and r_{2y} - Coordinates of point R2.
2. r_{3ax} and r_{3ay} - Coordinates of point R3'.
3. $\alpha_1, \alpha_2, \alpha_3$ - Angles for the left side mechanism.

Initial Estimates:

- $r_{2x} = -7, r_{2y} = -2$
- $r_{3ax} = 0.5, r_{3ay} = -0.5$
- $\alpha_1 = -30 \times \frac{\pi}{180}$ (radians), $\alpha_2 = -50 \times \frac{\pi}{180}$ (radians), $\alpha_3 = -127 \times \frac{\pi}{180}$ (radians)

Objective Function:

$$F = \sqrt{\sum_{i=1}^3 (fx(i)^2 + fy(i)^2)}$$

Right Side Optimization Problem

Objective: Minimize the error in the mechanism's path on the right side.

Decision Variables:

1. r_{4x} and r_{4y} - Coordinates of point R4.
2. r_{3bx} and r_{3by} - Coordinates of point R3".
3. $\gamma_1, \gamma_2, \gamma_3$ - Angles for the right side mechanism.

Initial Estimates:

- $r_{4x} = -5.6, r_{4y} = 0.5$
- $r_{3bx} = -2, r_{3by} = -0.5$
- $\gamma_1 = -78 \times \frac{\pi}{180}$ (radians), $\gamma_2 = -50 \times \frac{\pi}{180}$ (radians), $\gamma_3 = -191 \times \frac{\pi}{180}$ (radians)

Objective Function:

$$G = \sqrt{\sum_{i=1}^3 (gx(i)^2 + gy(i)^2)}$$

Objective Function for the Left Side (F):

For the left side, the objective function F is calculated based on the discrepancies between the expected positions (defined by the input precision points) and the positions achieved by the mechanism given its current configuration (determined by the variables $r2x, r2y, r3ax, r3ay, \alpha_1, \alpha_2, \alpha_3$).

The discrepancies in positions for each precision point are calculated using the following equations:

For $i = 1$ to 3 (since there are three precision points):

$$fx(i) = r2x + r3ax + \text{inputs.deltax}(i) - r3ax\cos(\alpha(i)) + r3aysin(\alpha(i)) - r2x\cos(\text{inputs.beta}(i)) + r2ysin(\text{inputs.beta}(i))$$

$$fy(i) = r2y + r3ay + \text{inputs.deltay}(i) - r3ay\cos(\alpha(i)) - r3ax\sin(\alpha(i)) - r2y\cos(\text{inputs.beta}(i)) - r2x\sin(\text{inputs.beta}(i))$$

Then, the objective function F is the square root of the sum of the squares of all these discrepancies:

$$F = \sqrt{\sum_{i=1}^3 (fx(i)^2 + fy(i)^2)}$$

Objective Function for the Right Side (G):

Similarly, for the right side, the objective function G is calculated based on the discrepancies between the expected positions and the positions achieved by the mechanism's right side, given its configuration (determined by the variables $r4x, r4y, r3bx, r3by, \gamma_1, \gamma_2, \gamma_3$).

For $i = 1$ to 3:

$$gx(i) = r4x + r3bx + \text{inputs.deltax}(i) - r3bx\cos(\gamma(i)) + r3bysin(\gamma(i)) - r4x\cos(\alpha(i)) + r4ysin(\alpha(i))$$

$$gy(i) = r4y + r3by + \text{inputs.deltay}(i) - r3by\cos(\gamma(i)) - r3bx\sin(\gamma(i)) - r4y\cos(\alpha(i)) - r4x\sin(\alpha(i))$$

And the objective function G is defined as:

$$G = \sqrt{\sum_{i=1}^3 (gx(i)^2 + gy(i)^2)}$$

In both cases, F and G aim to minimize these discrepancies, ideally driving them towards zero, which would indicate that the mechanism's paths align perfectly with the desired precision points.

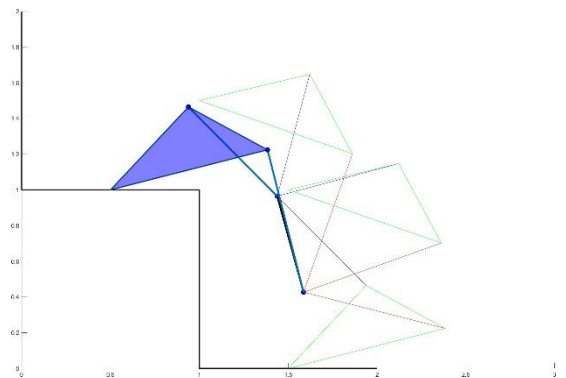


Figure 3 Path Generation Animation

LFval	RFval	r1	r2	r3	r4	alpha1	alpha2	alpha3	gamma1	gamma2	gamma3
0.00001540	0.00000448	2.03882579	0.70710938	0.89078820	2.26741122	-33.30303172	-33.30280622	0.00003877	305.75218704	246.10590233	191.85860057

Square root of a negative number encountered. Mechanism configuration might be incorrect.

2.3 Function Generation

Objective: Minimize the objective function that quantifies the discrepancy between the desired and the actual positions of the mechanism at various precision points.

Decision Variables:

1. r_3x and r_3y - Coordinates of point R3.
2. r_4x and r_4y - Coordinates of point R4.
3. ϕ_1, ϕ_2, ϕ_3 - Angles defining the orientation of the links at the precision points.

Initial Values

The initial values provided in your code are:

- $r_3x = -2, r_3y = 4$
- $r_4x = -3, r_4y = 5$
- $\phi_1 = 20^\circ, \phi_2 = 30^\circ, \phi_3 = 40^\circ$

Objective Function (G)

The objective function G is designed to minimize the sum of squared differences between the desired and computed positions at the precision points. It is defined as:

$$G = \sum_{i=1}^3 (fx(i)^2 + fy(i)^2)$$

where:

- $fx(i)$ and $fy(i)$ represent the discrepancies in the x and y directions for the i th precision point.

Specifically, for each precision point:

$$fx(i) = -r_2x \cdot (\cos(\text{inputs.beta}(i)) - 1) + r_2y \cdot \sin(\text{inputs.beta}(i)) - r_3x \cdot (\cos(\phi(i)) - 1) + r_3y \cdot \sin(\phi(i)) + r_4x \cdot (\cos(\text{inputs.gamma}(i)) - 1) - r_4y \cdot \sin(\text{inputs.gamma}(i))$$

$$fy(i) = -r_2y \cdot (\cos(\text{inputs.beta}(i)) - 1) - r_2x \cdot \sin(\text{inputs.beta}(i)) - r_3y \cdot (\cos(\phi(i)) - 1) - r_3x \cdot \sin(\phi(i)) + r_4y \cdot (\cos(\text{inputs.gamma}(i)) - 1) + r_4x \cdot \sin(\text{inputs.gamma}(i))$$

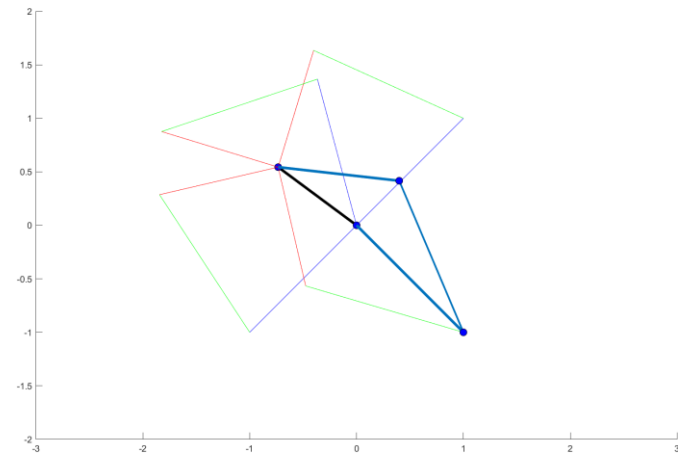


Figure 4 Function Generation

Fval	r1	r2	r3	r4	phi1	phi2	phi3
0.00000279	1.98081262	1.41421356	2.60128971	1.26900174	26.32910835	-14.38330215	10.46123513

Figure 5 Objective function values

Conclusion

In this lab, we explored the application of various optimization methods to solve different types of problems, demonstrating the nuanced performances of these methods depending on the nature of the problem and the chosen starting points.

In the first part of the lab, we focused on optimizing Himmelblau's function, a well-known test case for optimization algorithms. We employed three different methods: the steepest descent method, Newton's method, and MATLAB's built-in function **fminunc**. Our findings revealed that while the steepest descent method eventually found the final estimate points, it required a significant number of iterations, indicating a slower convergence rate. On the other hand, Newton's method was more efficient, reaching the final estimate points in fewer iterations. However, its performance was highly sensitive to the initial points, leading to convergence to local maxima in our experiments, underscoring the importance of initial value selection in optimization processes.

The **fminunc** function showcased a balanced approach, efficiently finding the final estimate points and providing insight into the optimal number of iterations needed for convergence. This illustrates the advantages of leveraging built-in functions in MATLAB, which are typically optimized for performance and robustness.

In the second part, the lab shifted focus to a mechanical design problem, employing the given algorithm and **fminunc()** for motion and path generation. While the optimization algorithms successfully generated solutions, the resulting outputs were imaginary, indicating non-feasible solutions for the function generation task. This outcome highlights a critical aspect of optimization in engineering design: the feasibility and physical realizability of solutions are as crucial as the mathematical optimality. In conclusion, this lab not only provided a hands-on experience with different optimization methods but also emphasized the critical thinking required to choose an appropriate method based on the problem at hand. It underscored the significance of initial conditions in the context of local and global optima and demonstrated the practical challenges of translating mathematical solutions into feasible engineering designs.

References

Lab2 Manual

ChatGpt